Exp 31 :- CMAC subkey Generation

Aim :- To write a C program to generate CMAC subkeys K_1 and K_2

Algorithm :-

1. Initalize the block with all zero bits
2. Apply the block cipher to obtain L
3. left shift L by one bit to generate K1
4. XOR with the constant when the most significant bit is 1
5. Repeat the process with K1 to generate K2

Output :- Enter L in hexadecimal : 123456789ABCDEF0

L = 123456789ABCDEF0

K1 = 2468ACF13579BDE0

K2 = 48D159E26AF37BC0

Result :- Thus, the program was Executed successfully

Exp 32 :- DSA signature

Aim:- To write a C program to demonstrate that DSA signature can differ when the same message is signed multiple time because a new random value k is used.

Algorithm :-

1. Read the message
2. Generate a random value k DSA signature
3. Generate another random k and sign the same message again.
4. Compare the two signature

Output:- message = 25

First Sign : k = 34

(r, s) = (59, 42)

Second Sign = k = 71

(r, s) = (96, 52)

Signatures are different.

Result:- Thus, the program executed successfully.

Exp 33 :- DES Implimentation

Aim:- To Implement the data encryption standard (DES) Algorithm for encryption and decryption using 64-bit block and 56-bit effective key.

Algorithm :-

1. Read the 64-bit plaintext and 64-bit keys.
2. Apply the Initail permutations and divide the block into two halves.
3. Perform 16 feisel rounds using generated subkeys.
4. Apply the final permutations to obtain ciphertext

Output :-

Plaintext : 0000000100100011

key : 0111111111101

Cipher text : 1110100000101 1

Decrypted : 0000001001000 11

Result :-

Thus, the program was executed successfully.

Exp. 34 : Padding

Aim :- To implement padding for ECB, CBC and CFB modes.

Algorithm :- Read the plaintext

2. Check the block size
3. Add , followed by as if needed.
4. Add a full padding block when the message is already complete.
5. Display the padded message.

Output :- HELLO

Padded data : HELLO100

Result :- Thus, the program was executed successfully.

Exp 35 One - Time Pad Vigenere

Aim:- To implement one - time pad version of the vigenere cipher.

Algorithm:-

1. Read the plaintext
2. Read the key stream
3. Convert letters to numbers
4. Calculate $C = (P+k) \bmod 26$
5. Display the ciphertext
6. STOP

Output:-

Plaintext : SEND MORE MONEY

key : 9 0 1 7 23 15 21 14 11 11 2 8 9

Ciphertext : BEOK JMQS XGNEY

Result:- Thus, the program was executed successfully.

Exp 36 Affine Caesar Cipher.

Aim :- To Implement the affine caesar cipher using $C = (ap + b) \bmod 26$.

Algorithm :-

1. Read plaintext a and b
2. Convert each letter to a number
3. Calculate $(a * p + b) \bmod 26$
4. Convert the result back to a letter.
5. Display ciphertext
6. STOP

Output :-

Plaintext : HELLO

a b : 5 8

Cipher text : RCLLA

Result :-

Thus, the program was executed successfully.

Exp 37: MonoAlphabetic Frequency Attack.

Aim:- To perform frequency analysis on a mono-Alphabetic substitution cipher.

Algorithm:-

1. Read the cipher text
2. count each letters frequency
3. Arrange letters by frequency.
4. Display the top 10 letters.
5. Use frequency order to estimate plaintext.

Output:-

WKLV LV D WHVW PITVVDJH

TOP 10 frequencies:

V=4
W=3
H=3
L=2
D=2

Result:-

Thus, the program was executed successfully.

Exp 38: Hill cipher known plaintext attack

Aim

To demonstrate recovery of a Hill cipher key using known plaintext ciphertext pairs.

Algorithm

1. Read two plaintext blocks
2. Read their corresponding ciphertext blocks.
3. from plaintext and ciphertext matrices.
4. calculate the inverse of the plaintext matrix
5. obtain the hill cipher key matrix.

Output

plaintext matrix : 1 2
3 5

ciphertext matrix: 5 8
13 21

Recovered key matrix: 3 4
5 7

Result:

Thus the program was executed successfully.

Exp 39:- Additive Cipher Frequency Attack.

Aim:- To perform a frequency attack on an additive cipher and display the top 10 possible plaintext.

Algorithm:-

1. Read the ciphertext
2. Try all 26 possible keys
3. Decrypt using each key.
4. Display the possible plaintexts.
5. Arrange likely results first.
6. STOP

Output:- KHOOR

Top 4 possible plaintexts.

key 0 : KHOOR
key 1 : JGNNQ
key 2 : IFMMP
key 3 : HELLO
key 4 : GDKKN

Result:- Thus, the program was executed successfully.

Exp 4D : MonoAlphabetic Frequency attack.

Aim:- To perform frequency analysis on a monoalphabetic substitution cipher.

Algorithm:-

1. Read the ciphertext
2. Count the frequency of each letter
3. Sort the letters by frequency
4. Display the top 10 possible frequency matches.
5. Use the frequency pattern to estimate plaintext.
6. STOP

Output:-

WKLV LV O

TOP 5 possible letters:

V → freq 4
W → freq 3
H → freq 3
L → freq 2
D → freq 2

Result:-

Thus the program was executed successfully.